Comparaison C++ vs Python

Vérification que l'implémentation C++ (`final/`) est cohérente avec une implémentation Python équivalente, et comparaison de performance (vitesse, mémoire).

Méthode

- **C++** : `final/main.cc` + `final/src/{pretreatment,ml}.cc`, HOG (pyramide 2 niveaux) + histogramme couleur HSV (OpenCV), Random Forest from-scratch.
- **Python** : `final/scripts/train_python_rf.py`, pipeline **indépendant** (n'utilise pas le C++), réplique la même logique de features (HOG pyramide via `skimage`, histogramme HSV) et entraîne un `sklearn.ensemble.RandomForestClassifier`.
- **Hyperparamètres identiques** : 50 arbres, profondeur max 14, min 2 échantillons par feuille, `max_features = sqrt(n_features)`, seed 42.
- **Données** : dataset Kaggle `gpiosenka/butterfly-images40-species` (100 classes), 12594 images d'entraînement (`train/`), 500 de test (`test/`).
- **Build C++** : `cmake -B build -DCMAKE_BUILD_TYPE=Release` (sans sanitizers : `-O3` seul ; `-fsanitize=address,undefined` gonfle fortement la mémoire mesurée et ne reflète pas un usage réel).

Vérification de cohérence

Les deux implémentations produisent des vecteurs de features de **taille identique** ($3624 = 2 \times 1764$ HOG + 96 histogramme couleur) pour une même image, confirmant que la logique de prétraitement est bien répliquée à l'identique côté Python.

	C++ (from scratch)	Python (`sklearn.RandomForestClassifier`)
Top-1 Accuracy	31.2 %	28.4 %
Top-5 Accuracy	53.8 %	48.8 %

Les deux implémentations, entraînées sur les mêmes données avec les mêmes hyperparamètres et la même logique de features, obtiennent des scores dans le même ordre de grandeur (écart de quelques points, cohérent avec des détails d'implémentation différents : tie-breaking sur les seuils de split, ordre du bootstrap, implémentation HOG OpenCV vs `skimage` légèrement différente au niveau du calcul des gradients). C'est cohérent avec le fait qu'il s'agit du même algorithme (CART + bagging), pas d'une divergence de logique.

Comparaison de performance

`final/main.cc` chronomètre désormais séparément l'extraction des features et l'entraînement/prédiction du Random Forest (`extraction_seconds`, `train_seconds`, `predict_seconds`), ce qui permet une comparaison **RF contre RF** et **extraction contre extraction**, plutôt qu'un temps global qui mélangeait les deux.

Attention : l'extraction C++ est **séquentielle** (une image à la fois), alors que l'extraction Python (`joblib`, non chronométrée séparément dans le script) tourne **en parallèle sur tous les cœurs** : la comparaison d'extraction n'est donc indicative que du narrow-case "un seul cœur" pour le C++.

	C++ (from scratch, `-O3`, 1 thread)	Python (`sklearn`, `n_jobs=1`)
Extraction (12594 images train)	7.03 s	non isolée (parallèle, `joblib`)
Entraînement RF (`fit`)	115.86 s	39.65 s
Extraction (500 images test)	0.49 s	non isolée (parallèle, `joblib`)
Prédiction (`predict`)	0.013 s	0.011 s
Pic mémoire (pipeline `train` complet)	2.15 Go (RSS, process complet)	637 Mo (process principal ; exclut les workers `joblib` parallèles)

Lecture : - **Entraînement RF** : sklearn est ~**2.9x plus rapide** que notre implémentation from-scratch (39.65 s vs 115.86 s), pour un entraînement strictement équivalent (mêmes données, mêmes hyperparamètres). sklearn utilise un tri incrémental optimisé à chaque split, alors que notre C++ re-trie les indices depuis zéro à chaque candidat de split (`O(n log n)` par feature par nœud). C'est la piste d'optimisation la plus impactante si on voulait rapprocher les deux. - **Prédiction** : quasi identique des deux côtés (13 ms vs 11 ms sur 500 images), aucune différence significative. - **Mémoire** : le pic C++ (2.15 Go) couvre tout le pipeline "train" en un seul process (chargement OpenCV de 12594 images + extraction + stockage des features + arbres) ; celui du Python (637 Mo) est sous-estimé par rapport au travail réellement effectué, puisque l'extraction des features tourne dans des processus enfants `joblib` non comptabilisés par `resource.getrusage(RUSAGE_SELF)`.

Reproduire

```
# C++
cd final
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build
/usr/bin/time -v ./build/butterfly_rf train <dataset>/train/ model.rf
/usr/bin/time -v ./build/butterfly_rf test <dataset>/test/ model.rf

# Python
uv run final/scripts/train_python_rf.py predictions_python_rf.csv
```